JAWABAN UJIAN AKHIR SEMESTER (UAS)

IT Platform and Infrastructure - Semester Genap 2025/2026

Nama	Muhammad Abiyyu Rahman
NIM	25.51.1738
Program Studi	S2 Informatika, Fakultas Ilmu Komputer
Mata Kuliah	MT107 - IT Platform and Infrastructure
Dosen Pengampu	I Made Artha Agastya, S.T., M.Eng., PhD. / Dr. Ferry Wahyu Wibowo, S.Si., M.Cs.
Jenis Ujian	UAS Semester Genap 2025/2026 - Online Waskita (Terbuka)

BAGIAN A: ARSITEKTUR & OPTIMASI PENYIMPANAN

Soal 1 CSV vs Apache Parquet

Pertanyaan A: Mengapa CSV membebani CPU dan RAM pada skala Terabyte

Secara anatomi internal, CSV adalah format berbasis baris (row-oriented) dan berbasis teks (text-encoded) tanpa metadata maupun skema. Karakteristik ini menimbulkan empat beban utama saat data mencapai skala Terabyte:

- **Parsing berbasis karakter.** Setiap byte harus dipindai untuk mendeteksi delimiter, tanda kutip, dan newline. Pada 2 TB data, CPU melakukan miliaran operasi perbandingan karakter hanya untuk memisahkan field, sebelum komputasi analitik apa pun dimulai.
- **Tidak ada skema tertanam.** Semua nilai tersimpan sebagai string. Angka seperti 1250000 harus dikonversi (type casting/inference) ke integer atau double pada saat runtime, per nilai per baris — beban CPU murni yang berulang setiap kali data dibaca.
- **Representasi teks yang boros.** Angka 64-bit yang dalam biner hanya 8 byte dapat memakan belasan byte sebagai teks. Saat dideserialisasi menjadi objek di heap JVM, ukuran in-memory dapat membengkak 3–5 kali lipat dari ukuran on-disk, sehingga 2 TB CSV dengan mudah melampaui kapasitas RAM agregat klaster dan memicu Out Of Memory (OOM).
- **Pembacaan seluruh baris (full row scan).** Karena orientasinya baris, walaupun query hanya membutuhkan 2 kolom dari 50 kolom, seluruh baris tetap harus dibaca dari disk, ditransfer melalui jaringan, dan di-parse. I/O dan memori terbuang untuk 48 kolom yang tidak pernah dipakai.

Kombinasi keempat faktor tersebut menjelaskan mengapa pipeline crash pada CSV 2 TB, sementara pada Parquet 400 GB berjalan lancar: volume byte yang benar-benar menyentuh CPU, jaringan, dan RAM jauh lebih kecil.

Pertanyaan B: Mekanisme Projection Pushdown dan Predicate Pushdown pada Parquet

Parquet adalah format kolumnar yang membagi file menjadi row groups (umumnya 128 MB), di mana setiap row group menyimpan data per kolom dalam column chunks, dan footer file menyimpan metadata statistik (min/max, null count) untuk setiap chunk.

- **Projection Pushdown (seleksi kolom).** Saat query hanya membutuhkan, misalnya, kolom user_id dan amount, reader Parquet membaca footer terlebih dahulu, mengetahui offset byte setiap column chunk, lalu hanya membaca chunk milik dua kolom tersebut. Kolom lain tidak pernah

dibaca dari disk maupun dikirim lewat jaringan. Pada tabel 50 kolom, I/O berkurang hingga ±96%.

- **Predicate Pushdown (seleksi baris via statistik).** Filter seperti WHERE tanggal = '2026-07-01' dievaluasi terlebih dahulu terhadap statistik min/max di metadata setiap row group. Jika rentang min–max sebuah row group tidak memuat nilai filter, seluruh row group di-skip tanpa satu byte pun dibaca. Eliminasi terjadi sebelum I/O, bukan setelah data masuk memori.

Dampak gabungan pada kasus di atas: dari 400 GB Parquet, data yang benar-benar dibaca ke memori klaster bisa hanya berukuran beberapa GB (kolom relevan × row group relevan). Beban I/O jaringan turun drastis, jumlah objek di heap JVM mengecil, tekanan Garbage Collection menurun, dan pipeline terbebas dari OOM. Ditambah encoding kolumnar (dictionary, RLE) dan kompresi Snappy per kolom, rasio penyusutan 2 TB menjadi 400 GB menjadi wajar karena data sejenis dalam satu kolom sangat mudah dikompresi.

Soal 2 Pemilihan Kolom Partisi Fisik

Kolom yang paling tepat adalah **Tanggal_Transaksi (365 nilai unik)**, bukan User_ID (10.000.000 nilai unik).

Analisis Risiko Over-partitioning (Small File Problem) pada User_ID

- **Ledakan jumlah direktori dan file.** partitionBy menciptakan satu direktori fisik per nilai unik. User_ID menghasilkan 10 juta direktori; jika setiap job menulis dari ratusan task, jumlah file bisa mencapai ratusan juta file berukuran KB.
- **Beban metadata (NameNode/metastore).** Setiap file dan direktori adalah entri metadata yang harus disimpan di memori NameNode HDFS atau di-list oleh object store. Listing 10 juta partisi saja dapat memakan waktu lebih lama daripada pemrosesan datanya.
- **Overhead scheduling task.** Spark membuat minimal satu task per file kecil. Jutaan task berdurasi milidetik membuat overhead scheduling dan serialisasi jauh melampaui waktu komputasi efektif.
- **Throughput I/O runtuh.** File berukuran jauh di bawah block size 128 MB menghilangkan keuntungan sequential read; disk dan jaringan didominasi oleh open/close file dan seek, bukan transfer data.

Sebaliknya, Tanggal_Transaksi menghasilkan 365 partisi berukuran besar dan seimbang (±4,4 GB per hari untuk 1,6 TB/tahun), selaras dengan pola query analitik yang hampir selalu memfilter berdasarkan rentang tanggal — sehingga partition pruning bekerja efektif. Untuk kebutuhan lookup per pengguna pada kolom ber-kardinalitas tinggi seperti User_ID, teknik yang tepat adalah *bucketing* atau Z-ordering, bukan partisi fisik direktori.

BAGIAN B: MATEMATIKA KLASTER & ESTIMASI RESOURCE

Soal 3 Matematika Partisi Data & Optimalisasi I/O

1) Ukuran data setelah konversi ke Parquet

Ukuran Parquet = 25% × 1.600.000 MB = 400.000 MB (400 GB)

2) Jumlah partisi ideal (aturan 128 MB per partisi)

N partisi = 400.000 MB ÷ 128 MB = 3.125 partisi

Jadi konfigurasi yang benar adalah spark.sql.shuffle.partitions = 3125 agar setiap task memproses tepat ±128 MB.

3) Analisis konfigurasi default (200 partisi)

Ukuran rata-rata per partisi = 400.000 MB ÷ 200 = 2.000 MB = 2 GB per partisi

Analisis: 2 GB per partisi berarti ±15,6 kali lipat dari ukuran ideal 128 MB. Perlu diingat bahwa 2 GB tersebut adalah ukuran terkompresi on-disk; saat dideserialisasi menjadi objek Java di heap JVM, footprint aktual dapat membengkak menjadi 6–10 GB per task. Konsekuensinya:

- **Risiko OOM tinggi.** Satu task harus menampung seluruh partisi di memori executor. Jika executor memory (dikurangi overhead dan fraksi storage) tidak mampu menampung objek sebesar itu, task gagal dengan java.lang.OutOfMemoryError, dan retry akan gagal dengan pola yang sama.
- **GC Overhead.** Kalaupun muat, heap yang nyaris penuh oleh objek raksasa memaksa JVM melakukan Full GC berulang. Jika waktu GC melebihi 98% waktu eksekusi dengan pemulihan heap < 2%, muncul error "GC overhead limit exceeded", dan sebelum itu pun throughput sudah anjlok karena executor lebih sibuk membersihkan memori daripada menghitung.

Kesimpulan: konfigurasi default 200 partisi **tidak aman** untuk beban 400 GB. Nilai yang benar adalah ±3.125 partisi (atau mengaktifkan Adaptive Query Execution agar Spark menyesuaikan jumlah partisi shuffle secara dinamis).

Soal 4 Alokasi Resource YARN & Kalkulasi Latensi Jaringan

Pertanyaan A: Perhitungan Alokasi Executor

Langkah 1 - Sisihkan resource OS per node.

Core tersedia per node = 24 - 1 = 23 core

RAM tersedia per node = 128 - 1 = 127 GB

Langkah 2 - Tentukan core per executor (batas maksimum 5 core).

--executor-cores = 5

Executor per node = ⌊23 ÷ 5⌋ = 4 executor per node

Langkah 3 - Total executor klaster dan pengurangan Application Master.

Total executor murni = 5 node × 4 = 20 executor

--num-executors = 20 − 1 (AM/Driver) = 19 executor efektif

Langkah 4 - Memori per executor (dikurangi 10% YARN Memory Overhead).

RAM per slot executor = 127 GB ÷ 4 = 31,75 GB

RAM bersih = 31,75 × (1 - 10%) = 28,575 GB → dibulatkan ke bawah = 28 GB

Jawaban akhir konfigurasi spark-submit:

```
spark-submit \
  --master yarn --deploy-mode cluster \
  --executor-cores 5 \
  --num-executors 19 \
  --executor-memory 28G
```

Ringkasan: (1) --executor-cores = 5; (2) total executor terbentuk = 20, executor efektif --num-executors = 19; (3) --executor-memory = 28 GB.

Pertanyaan B: Perhitungan Teoretis Waktu Shuffle

1) Rumus. Karena data terdistribusi merata dan seluruh node mentransfer secara paralel, waktu shuffle ditentukan oleh volume data per node dibagi bandwidth per node:

T_shuffle = (Total Data ÷ Jumlah Node) ÷ Bandwidth per Node

2) Perhitungan. Asumsi: 600 GB adalah total volume yang harus melewati jaringan, terbagi sempurna ke 5 worker node yang masing-masing bekerja pada batas penuh 125 MB/s.

Data per node = 600.000 MB ÷ 5 = 120.000 MB
T_shuffle = 120.000 MB ÷ 125 MB/s = 960 detik
960 detik = 16 menit 0 detik

Jadi waktu minimum teoretis fase Network Shuffle adalah 960 detik, atau 16 menit. Nilai ini adalah batas bawah ideal; pada praktiknya overhead serialisasi, disk spill, dan kontensi TCP membuat waktu aktual lebih lama. Angka ini juga menunjukkan bahwa pada klaster ber-bandwidth 1 Gbps, wide transformation adalah biaya terbesar pipeline — memperkuat pentingnya meminimalkan shuffle (broadcast join, partisi yang tepat).

BAGIAN C: DISTRIBUTED MACHINE LEARNING & TROUBLESHOOTING

Soal 5 Analisis Komparasi Model MLlib

1) Algoritma dengan jumlah Jobs kecil terbanyak dan dampaknya pada jaringan 100 Mbps

Logistic Regression. LR bersifat iteratif berbasis gradien: dengan maxIter = 50, setiap iterasi memicu minimal satu job agregasi (treeAggregate) untuk mengumpulkan gradien dari seluruh partisi ke driver, memperbarui vektor koefisien, lalu menyiarkannya kembali ke seluruh executor. Hasilnya adalah ±50 job kecil beruntun di Spark UI.

Dampak pada bandwidth 100 Mbps (±12,5 MB/s): setiap iterasi menuntut round-trip komunikasi driver ↔ executor. Latensi jaringan yang sempit membuat waktu tiap iterasi didominasi oleh fase komunikasi/sinkronisasi, bukan komputasi. Total waktu training memanjang secara linier terhadap jumlah iterasi. Random Forest sebaliknya menghasilkan job yang lebih sedikit namun berukuran besar, karena pembangunan pohon dilakukan level-per-level dengan agregasi statistik yang lebih jarang.

2) Algoritma dengan konsumsi RAM terbesar pada Storage Tab

Random Forest. Dengan numTrees = 100 dan maxDepth = 10, RF harus memelihara struktur data besar selama training: statistik kandidat split (histogram fitur) untuk banyak node pada 100 pohon sekaligus, cache node-ID yang memetakan setiap baris data ke posisi node di setiap pohon, serta sampel bootstrap (bagging). Struktur bantu ini di-persist di memori dan tampak menonjol pada Storage Tab. Model LR hanyalah satu vektor koefisien sepanjang jumlah fitur. Konsumsi memorinya untuk struktur model praktis dapat diabaikan.

3) Dampak penghapusan .persist() sebelum training Logistic Regression

Waktu eksekusi meledak drastis. Spark bersifat lazy dan bekerja berdasarkan lineage: tanpa persist, dataset 100 GB tidak disimpan di RAM, sehingga setiap job (setiap iterasi) memaksa Spark menghitung ulang seluruh rantai transformasi dan membaca ulang data dari sumbernya. Dengan 50 iterasi, data 100 GB dibaca/dikomputasi ulang hingga 50 kali. I/O total efektif ±5 TB. Karena LR adalah algoritma iteratif yang menyentuh dataset yang sama berulang kali, ia adalah use-case utama caching; menghapus

.persist() mengubah biaya baca data dari "sekali di awal" menjadi "berulang setiap iterasi", dan pada bandwidth 100 Mbps efeknya semakin parah.

Soal 6 Troubleshooting Produksi

Data observasi Spark UI (Tab Stages & Executors):

Worker Node	Jumlah Tasks	Waktu Rata-rata/Task	Shuffle Read	Status
worker-node-1	150	2 detik	40 MB	Succeeded
worker-node-2	150	3 detik	42 MB	Succeeded
worker-node-3	1	1,5 jam	18 GB	Running (Straggler)

Pertanyaan A: Masalah sistemik dan bukti indikatornya

Masalah yang terjadi adalah Data Skew (kemiringan distribusi data) yang menghasilkan straggler task. Satu kunci partisi "panas" (hot key) mendominasi hasil shuffle sehingga hampir seluruh data untuk kunci tersebut jatuh ke satu task di satu node. Bukti indikator dari tabel:

- **Ketimpangan Shuffle Read ekstrem:** worker-node-3 membaca 18 GB (18.432 MB) sementara node lain hanya 40-42 MB, selisih ±450 kali lipat, mustahil terjadi pada distribusi hash yang sehat.
- **Ketimpangan jumlah task:** node 1 dan 2 masing-masing menerima 150 task ringan, sedangkan node 3 hanya 1 task raksasa, pola khas seluruh baris ber-kunci sama terkumpul di satu partisi.
- **Ketimpangan durasi:** 2-3 detik vs 1,5 jam (±2.000×). Karena sebuah stage baru selesai setelah task terakhirnya selesai, seluruh klaster efektif menganggur menunggu satu straggler, utilisasi klaster runtuh.

Pertanyaan B: Penyelesaian permanen dengan Salting Technique

Prinsip: hot key dipecah secara artifisial dengan menambahkan suffix acak (salt) 0..N-1, sehingga baris-baris yang tadinya di-hash ke satu partisi tersebar merata ke N partisi dan diproses paralel oleh banyak executor. Pada operasi join, tabel pasangan (sisi kecil) direplikasi N kali agar setiap varian salted key tetap menemukan pasangannya.

Contoh implementasi PySpark:

```
from pyspark.sql import functions as F

SALT_FACTOR = 10  # hot key dipecah menjadi 10 sub-key

# 1. Sisi tabel BESAR (skewed): tambahkan salt acak 0..9 pada key
df_large_salted = df_large.withColumn(
    "salted_key",
    F.concat(F.col("key"), F.lit("_"),
             (F.rand() * SALT_FACTOR).cast("int").cast("string"))
)

# 2. Sisi tabel KECIL: replikasi setiap key sebanyak SALT_FACTOR kali
salt_array = F.array([F.lit(i) for i in range(SALT_FACTOR)])
df_small_salted = (
    df_small
    .withColumn("salt", F.explode(salt_array))
    .withColumn("salted_key",
                F.concat(F.col("key"), F.lit("_"),
                         F.col("salt").cast("string")))
)
```

```
# 3. Join pada salted_key -> beban 18 GB tersebar ke 10 task paralel
result = df_large_salted.join(df_small_salted,
                              on="salted_key", how="inner")

# (Untuk agregasi: groupBy salted_key terlebih dahulu,
#  lalu agregasi kedua pada key asli untuk menggabungkan hasil parsial)
```

Mekanisme di tingkat memori: hash partitioner kini menerima 10 nilai kunci berbeda untuk hot key yang sama, sehingga 18 GB terpecah menjadi ±1,8 GB per task yang tersebar ke banyak executor. Tidak ada satu heap JVM pun yang harus menampung seluruh beban, straggler hilang, tekanan GC menurun, dan durasi stage turun dari 1,5 jam mendekati waktu task terlama yang kini berukuran wajar. Solusi ini permanen karena tidak bergantung pada distribusi data mendatang. Hot key apa pun otomatis terpecah.
Catatan pelengkap (praktik modern): Spark 3.x menyediakan Adaptive Query Execution dengan spark.sql.adaptive.skewJoin.enabled = true, yang mendeteksi dan memecah partisi skew secara otomatis saat runtime. Salting tetap relevan sebagai solusi deterministik di tingkat kode, terutama untuk agregasi dan versi Spark lama.